



Compiler Design (UE23CS341B)

Chapter 1 – Introduction

Prakash C O

Department of Computer Science & Engineering

Compiler Design (UE23CS341B)

Chapter 1 – Introduction

1. History of Compilers
2. Types of compilers
3. The Language Processors
4. The phases of a compiler
5. Symbol Table management

Prakash C O

Department of Computer Science & Engineering



Compiler Design

➤ Course Info

➤ Why we need a Compiler?

➤ Why we need a Compiler?

- **Programming languages** are notations for describing computations to people and to machines.

High-level programming languages (like C, Java, Python) are designed for humans to easily read and write code.

- Machines do not understand programs in high-level programming languages. So, a software system is needed to do the translation. This is the **compiler**.

➤ History of Compilers

- ❑ **The first implemented compiler was written by Grace Hopper in 1952, for the A-0 System language.**



The term *compiler* was coined by Hopper. The A-0 functioned more as a loader or linker than the modern notion of a compiler.

- ❑ **The FORTRAN team led by John W. Backus at IBM introduced the first commercially available compiler, in 1957, which took 18 person-years to create.**
- ❑ **COBOL was the first programming language which was compiled on multiple platforms in 1960.**

The study of compiler design/writing touches upon

❑ Programming languages

- Compiler writers have to track new language features

❑ Machine architecture

- Compiler writers must take advantage of new hardware features

❑ Language theory,

- Compiler writing makes practical use of finite automata and push-down automata (scanning and parsing)

❑ Algorithms, and

- Compiler writers must take practical use of greedy algorithms (register allocation), heuristic search techniques (list scheduling), graph algorithms (dead-code elimination), dynamic programming (instruction selection),

❑ Software engineering.

- Working inside a compiler provides practical experience in software engineering that is hard to obtain with smaller, less intricate systems.

Why Compilers Are So Important?

▣ Interface Role

- Compilers provide an essential interface between **applications** and **machine architectures**.

▣ Theoretical Foundation

- They embody a wide range of **theoretical techniques** from computer science.

▣ Developer Training

- Studying compilers trains good developers.
- Compiler construction teaches **programming** and **software engineering skills**.

▣ Adaptability

- Machines have evolved continuously since their invention.
- **Changes in architecture ⇒ changes in compilers.**

▣ Problem-Solving Skills

- Learning compiler design teaches not only how to build compilers but also the **general methodology for solving complex and open-ended problems**.

Why Compilers Are So Important?

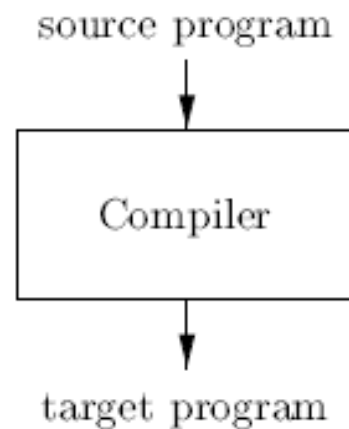
- **Compilation technology can be applied in many different areas**
 - **Binary translation**
 - Binary translation offers solutions for automatically converting executable code to run on new architectures without recompiling the source code
 - **Hardware synthesis**
 - A compiler that transforms programs written in a functional language (Haskell) into synchronous digital circuits, which we then synthesize onto an FPGA(field-programmable gate array) for prototyping purposes.
 - **Database query interpreters**
 - A query interpreter accepts a query and uses it to generate answers from the database.
 - A query compiler accepts a query and generates a program which, when run, yields the same result.

Language Processors

- ▣ **What is Compiler?**
- ▣ **Types of compilers**
- ▣ **What is Interpreter? How it is different from Compiler.**

Language Processors

- A **compiler** is a program that can read a program in one language — the source language — and **translate it into an equivalent program in another language** — the target language.
- An important role of the compiler is to report any errors in the source program that it detects during the translation process.



- If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs; see Fig. 1.2.

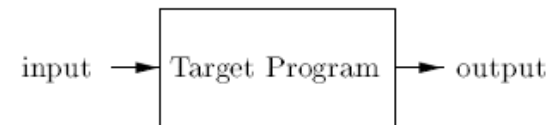


Figure 1 2 Running the target program

■ Types of compilers

- One pass compiler,
- Multi pass compiler,
- Cross compiler,
- Native Compiler,
- Decompiler.
- Source-to-source compiler,
- Self hosting compiler(Bootstrapping)
- Compiler-compiler

Language Processors

▣ **What is Interpreter? How it is different from Compiler.**

Language Processors

- **Interpreter:** Instead of producing a target program as a translation, an interpreter appears to **directly execute the operations specified in the source program on inputs supplied by the user**, as shown in Fig. 1.3.

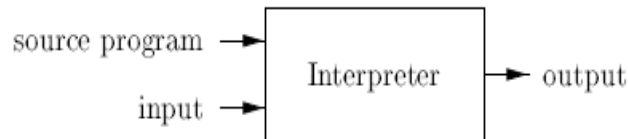


Figure 1 3 An interpreter

- The machine-language **target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs** .
- An **interpreter can usually give better error diagnostics** than a compiler, because it executes the source program statement by statement.
- **Examples** of programming languages that use interpreters: BASIC, Visual Basic, Python, Ruby, PHP, Perl, MATLAB, Lisp.

➤ Hybrid Compilers

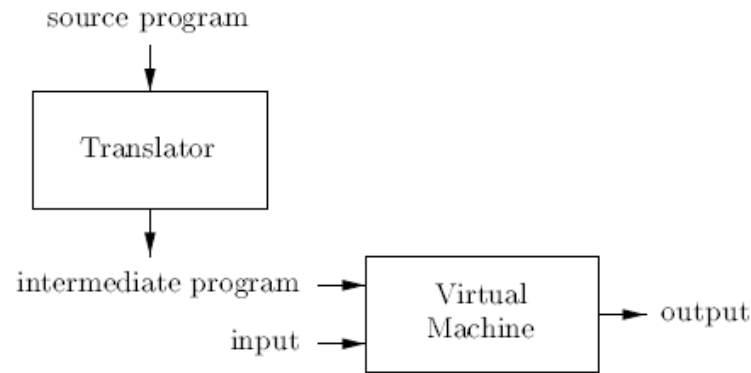


Figure 1.4 A hybrid compiler

- Java language processors combine compilation and interpretation, as shown in Fig. 1.4. A Java source program may first be compiled into an intermediate form called bytecodes. The bytecodes are then interpreted by a virtual machine.
- A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.
- In order to achieve faster processing of inputs to outputs, some Java compilers, called just-in-time (JIT) compilers, translate the bytecodes into machine language immediately before they run the intermediate program to process the input.

Compiler Is Not A One-Man-Show!

■ In addition to a compiler, several other programs may be required to create an executable target program, as shown in Fig. 1.5.

- **Preprocessor**
- **Assembler**
- **Linker/Loader**

These are the cousins of the compilers!

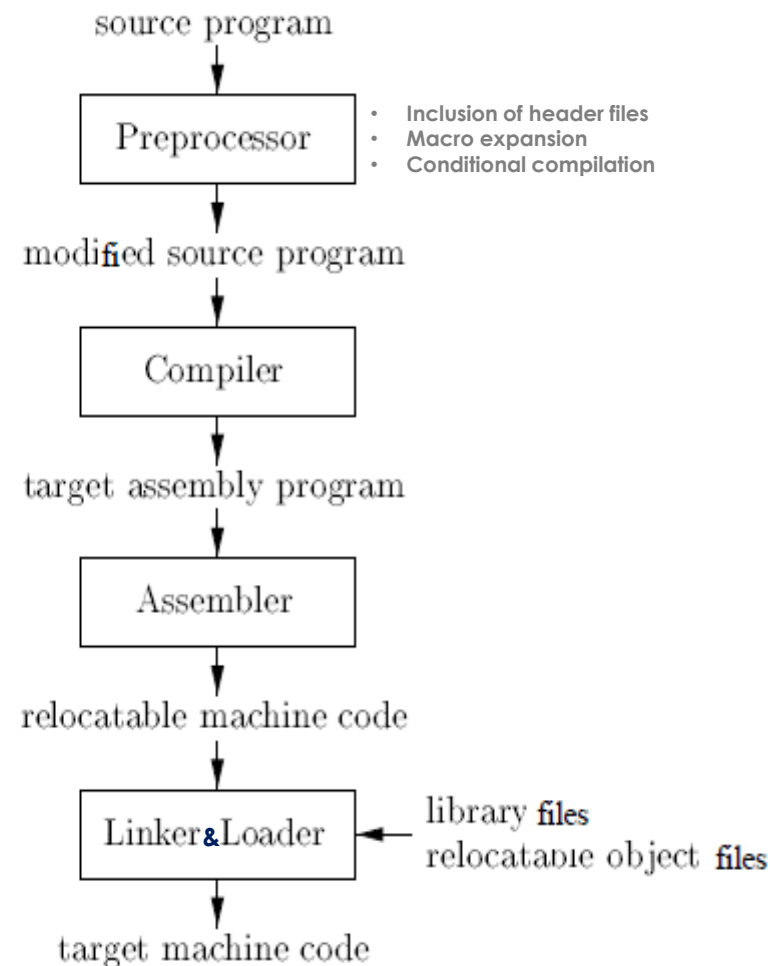


Figure 1.5 A language processing system

Why not letting the compiler generate machine code directly?

Language Processors

- A source program may be divided into modules stored in separate files.
 - The task of collecting the source program is done by a separate program, called a **preprocessor**.
 - The preprocessor may also expand shorthands, called **macros**, into source language statements.
- The modified source program is then fed to a compiler.

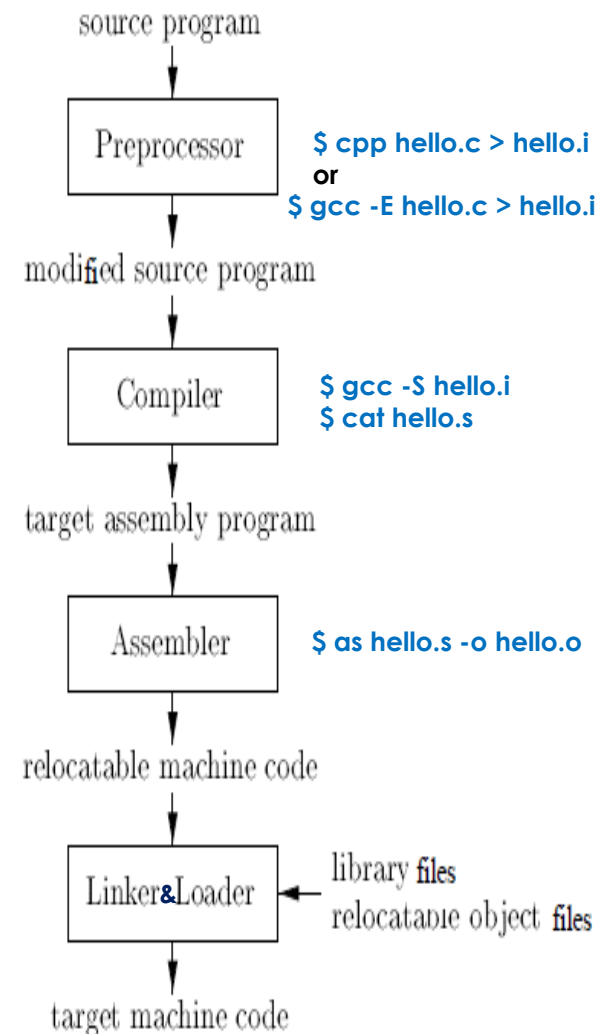


Figure 1.5 A language processing system

Language Processors

- The compiler may produce an **assembly language program** as its output, because assembly language
 1. is easier to produce as output
 2. is easier to debug
 3. makes code relocation much easier.
- The assembly language code is then processed by an **assembler** that produces **relocatable machine code** as its output.

Note: Developers and compiler writers can inspect the generated assembly to verify correctness, analyze performance, or debug issues.

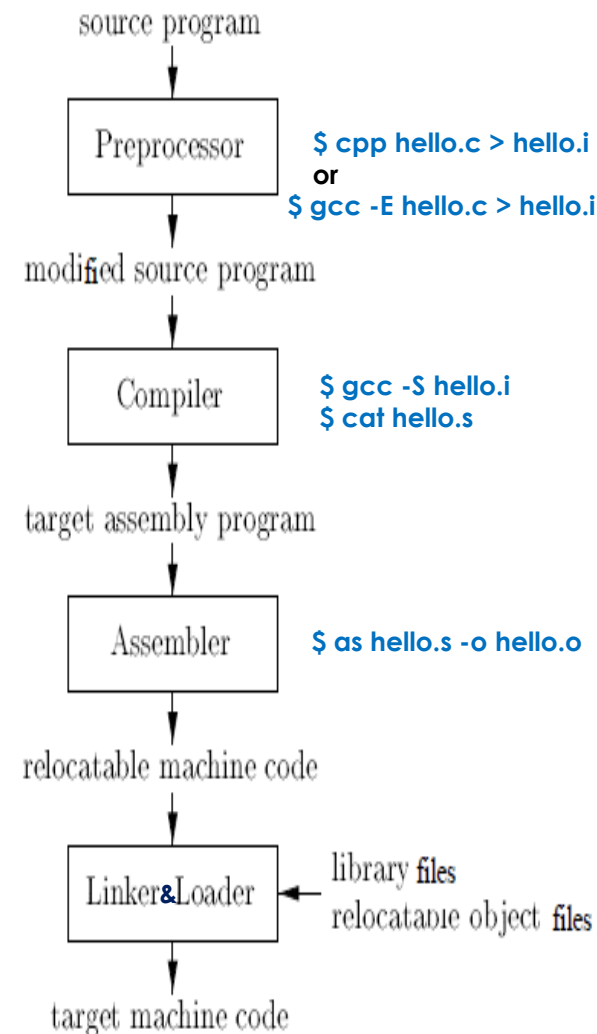


Figure 1.5 A language processing system

Language Processors

- ❑ Large programs are often compiled in pieces, so the **relocatable machine code** may have to be linked together with other **relocatable object files** and **library files** into the code that actually runs on the machine.
- ❑ The **linker** resolves external memory addresses, where the code in one file may refer to a location in another file.
- ❑ The **loader** then puts together all of the executable object files into memory for execution.
- ❑ **gcc demo**

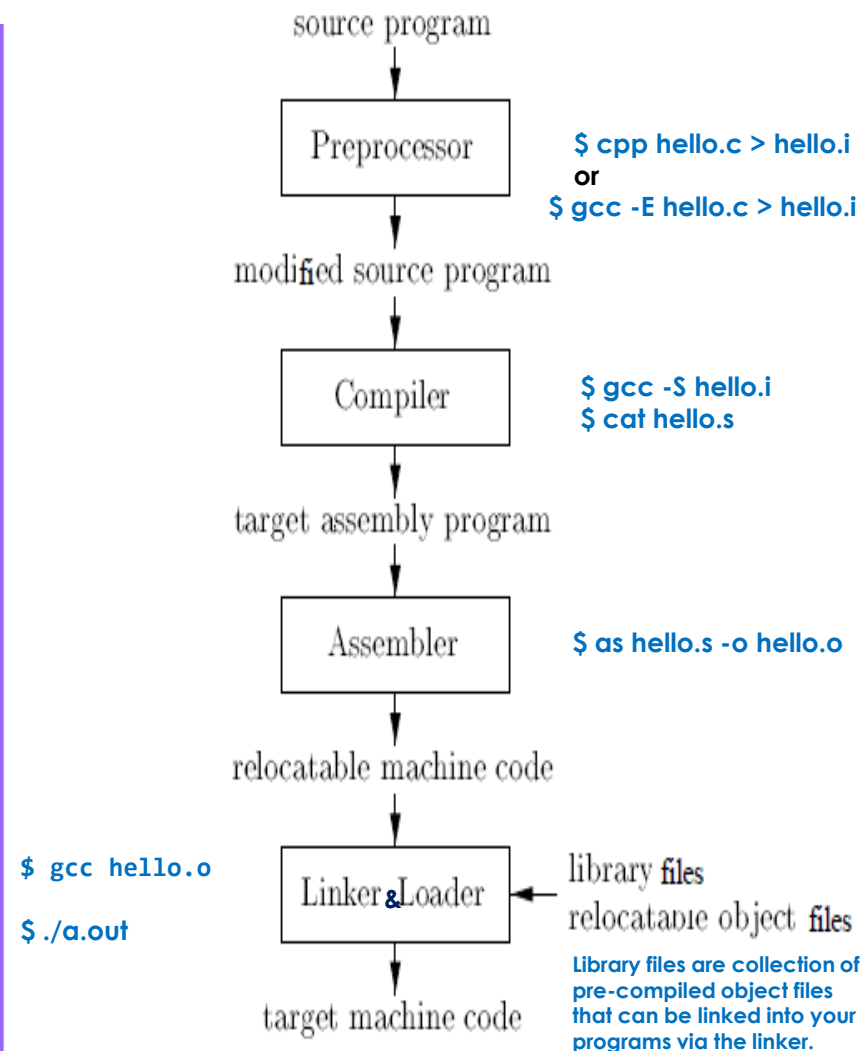
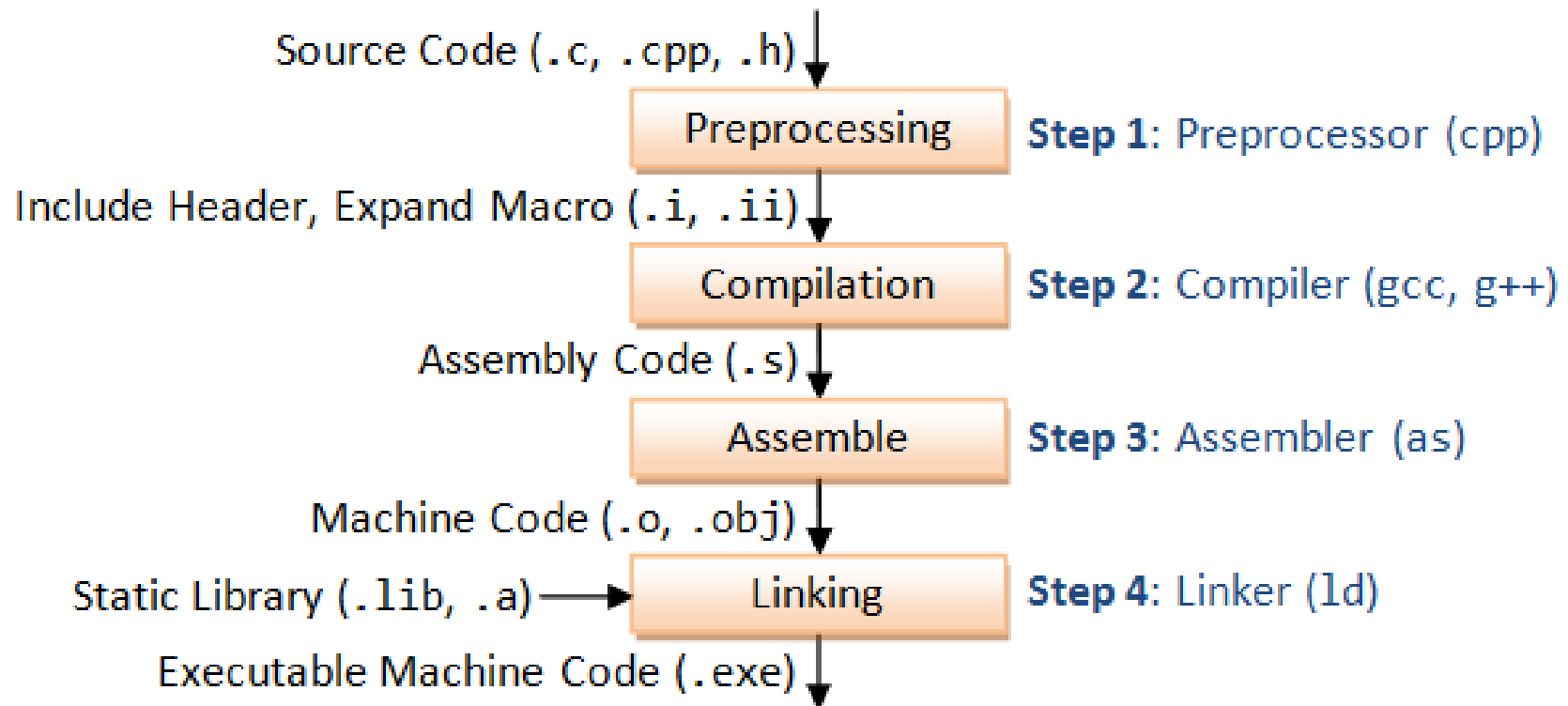


Figure 1.5 A language processing system

Language Processors

■ GCC Compilation Process



Note: '.i' for C programs and '.ii' for C++ programs

Source: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/gcc_make.html

Static Library and Shared library

- A **library** is a collection of pre-compiled object files that can be linked into your programs via the linker.

Examples are the system functions such as **printf()** and **sqrt()**.

- There are two types of external libraries: **static library** and **shared library**.

■ Static Library

- A **static library** has file extension of ".a" (archive file) in Unixes or ".lib" (library) in Windows.
- When your program is linked against a static library, the machine code of external functions used in your program is copied into the executable.
- A static library can be created via the archive program "ar.exe".

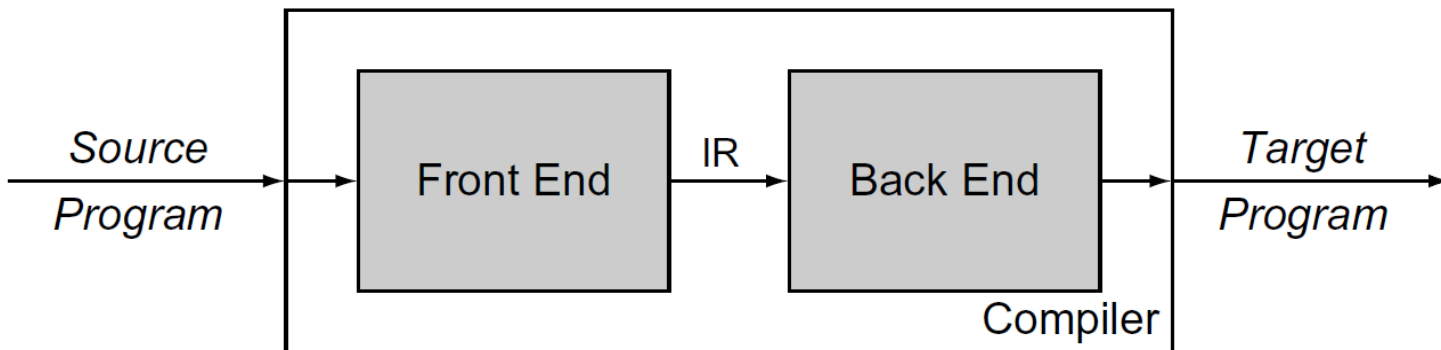
❑ Shared Library

- A shared library has file extension of ".so" (shared objects) in Linux or ".dll" (dynamic link library) in Windows.
- When your program is linked against a shared library, only a small table is created in the executable.
 - This table lists the names of the external functions and variables the program needs.
 - It also records which shared libraries (e.g., .so on Linux, .dll on Windows) provide them.
- Before the executable starts running, the operating system loads the machine code needed for the external functions - a process known as *dynamic linking*.
- The most operating systems allows one copy of a shared library in memory to be used by all running programs, thus, saving memory. The shared library codes can be upgraded without the need to recompile your program.
- ❑ **Dynamic linking** makes executable files smaller and saves disk space, because one copy of a library can be shared between multiple programs.

The Structure of a Compiler

- A compiler must both understand the source program that it takes as input and map its functionality to the target machine.

The distinct nature of these two tasks leads to a design that decomposes compilation into two major pieces: a *front end* and a *back end*.



- **The front end focuses on understanding the source-language program.**
- **The back end focuses on mapping programs to the target machine.**

The Structure of a Compiler

■ There are two parts in Compiler

■ Analysis Part (Front-end)

- Breaks up the source program into constituent pieces and imposes a grammatical structure on them.
- It then uses this structure to create an intermediate representation of the source program
- The analysis part also collects information about the source program and stores it in a data structure called a ***symbol table***, which is passed along with the intermediate representation to the synthesis part.

■ Synthesis Part (Back-end)

- Construct the desired target program from the intermediate representation and the information in the symbol table.

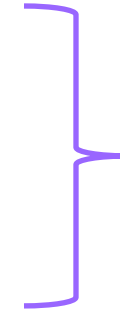
The Structure of a Compiler

Source Program



Front-end

Analysis Part
(aka **pass-1**)

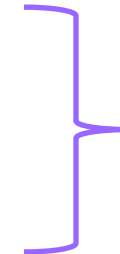


Intermediate code



Back-end

Synthesis Part
(aka **pass-2**)



Target Program



Symbol table



Figure: Two-Pass Compiler

The Structure of a Compiler

▣ Structure of Compiler

- ▣ If we examine the compilation process in more detail, it operates as a sequence of phases, each of which transforms one representation of the source program to another.

▣ Phases of a compiler

1. **Lexical Analysis**
2. **Syntax analysis**
3. **Semantic analysis**
4. **Intermediate Code Generator**
5. **Machine-Independent Code Optimizer**
6. **Code Generator**

Phases of a compiler

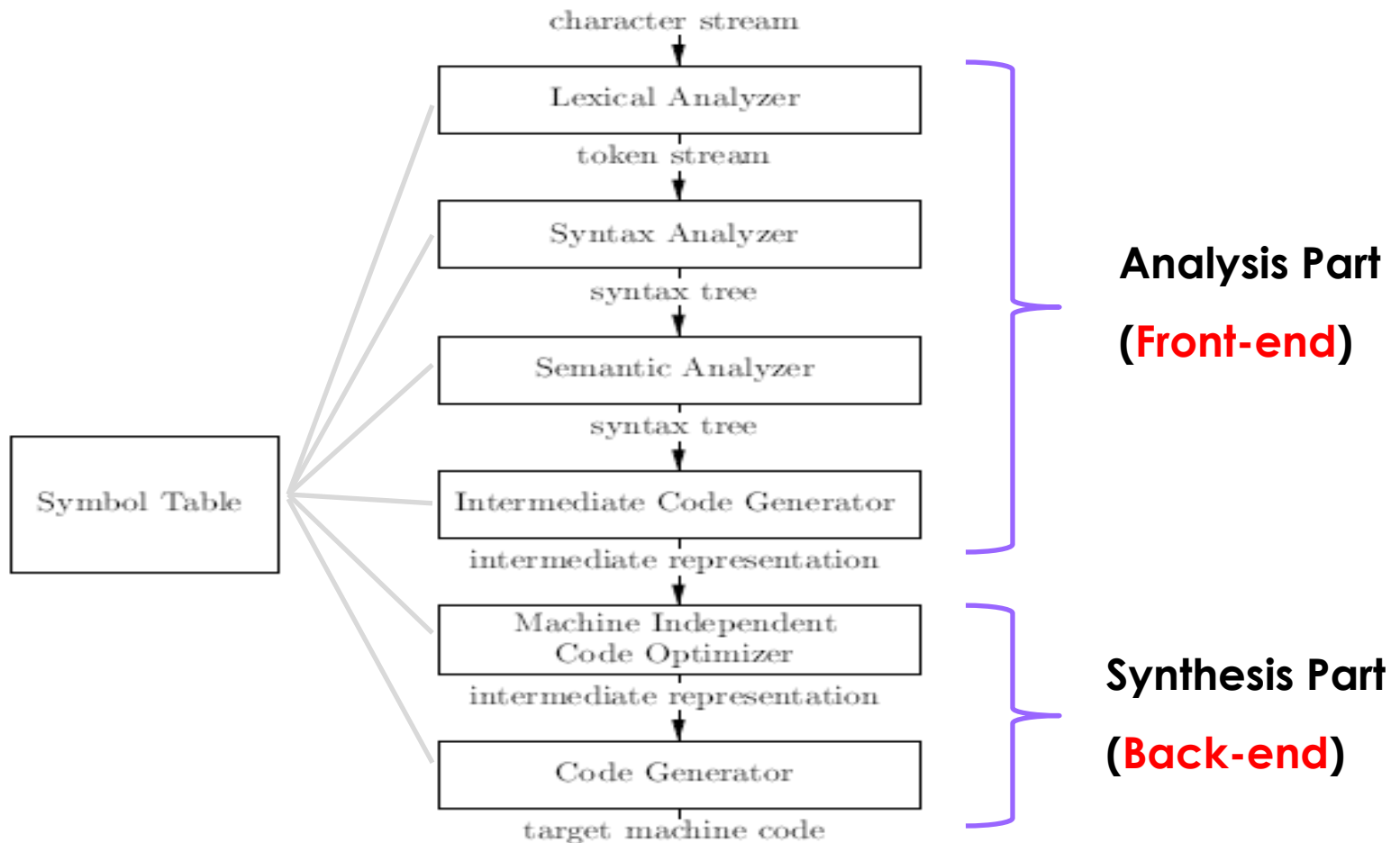


Figure: Phases of a compiler

Phases of a compiler

1. Lexical Analysis

- The first phase of a compiler is called *lexical analysis* or *scanning*.
- The lexical analyzer **reads the stream of characters** making up the source program and **groups the characters into meaningful sequences** called *lexemes*.
- Lexemes for the program statement **position = initial + rate * 60** are

<i>position</i>	}	identifiers
<i>initial</i>		
<i>rate</i>		
=	}	operators
+		
*		
60	}	literal

Lexeme types:

1. Keywords
2. Identifiers
3. Literals (aka Constants)
4. Operators
5. Punctuators

Phases of a compiler

1. Lexical Analysis

- For each lexeme, the lexical analyzer produces as output a *token of the form* **< token-name, attribute-value >** that it passes on to the subsequent phase, syntax analysis.
- In the token, the first component **token-name** is an **abstract symbol that is used during syntax analysis**, and the second component **attribute-value** (for identifiers) **points to an entry in the symbol table for this token**.

Lexemes

Position
=
initial
+
rate
*
60

Tokens (Implementation method-1)

Note: Attribute value is optional for keywords, operators and punctuation symbols

<id,1>
<=>
<id,2>
<+>
<id,3>
<*>
<60>

Symbol Table

	Name	Type	Size	Scope	...
1	position				
2	initial				
3	rate				
...	...				

Phases of a compiler

1. Lexical Analysis

- For each lexeme, the lexical analyzer produces as output a *token of the form* **< token-name, attribute-value >** that it passes on to the subsequent phase, syntax analysis.
- In the token, the first component **token-name** is an **abstract symbol that is used during syntax analysis**, and the second component **attribute-value** (for identifiers) **points to an entry in the symbol table for this token**.

Lexemes

Position
=
initial
+
rate
*
60

Tokens (Implementation method-2)

Note: Attribute value is optional only for keywords

<id,1>
<assign_op, = >
<id,2>
<arith_op, + >
<id,3>
<arith_op, *>
<num, 60>

Symbol Table

	Name	Type	Size	Scope	...
1	position				
2	initial				
3	rate				
...	...				

Phases of a compiler

1. Lexical Analysis

- **When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table.**
- **In certain pairs, especially operators, punctuation, and keywords, there is no need for an attribute value.**
- **Information from the symbol-table entry is needed for semantic analysis and code generation.**

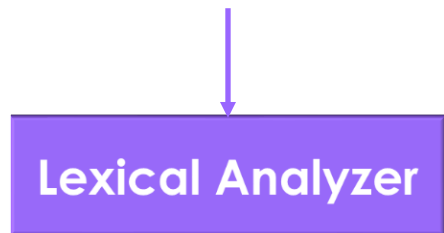
Phases of a compiler

1. Lexical Analysis

- Example: Lexical analysis of a program statement

position = initial + rate * 60

position = initial + rate * 60 (1.1)



Symbol Table

	Name	Type	Size	Scope
1	position			
2	initial			
3	rate			
...	...			

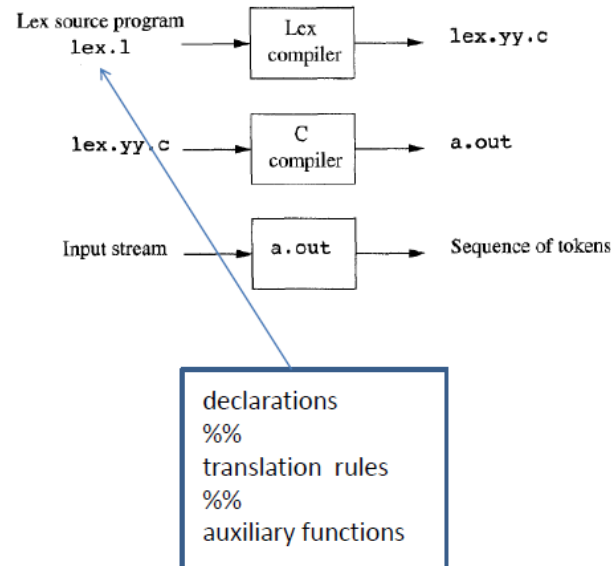
<id,1> <=> <id,2> <+> <id,3> <*> <60> (1.2)

Phases of a compiler

1. Lexical Analysis-Tools

□ LEX/FLEX

- **LEX/FLEX reads a specification file containing regular expressions and generates a C routine that performs lexical analysis.** Matches sequences that identify tokens.



2. Syntax Analysis

- The second phase of the compiler is ***syntax analysis*** or ***parsing***.
- Parsing is the process of determining how a written program statement or a string of terminals can be generated by a grammar.
- **Why syntax analysis ?**
 - We have seen that a lexical analyzer can identify tokens with the help of regular expressions and pattern rules.
But a lexical analyzer cannot check the syntax of a given program statement due to the limitations of the regular expressions.
 - Regular expressions cannot check balancing tokens, such as parenthesis.
Therefore, **syntax analysis phase uses context-free grammar (CFG)**, which is recognized by push-down automata.

Phases of a compiler

2. Syntax Analysis

Token examples: `<id,3>` , `<if>`, `<num, 60>`

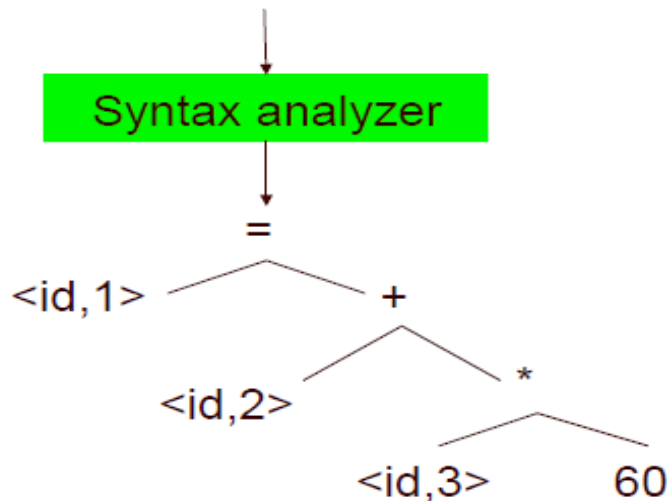
- The **parser** uses the **first components of the tokens** produced by the lexical analyzer **to create a tree-like intermediate representation** (i.e., hierarchical structure) **that depicts the grammatical structure of the token stream.**
- A typical **Intermediate Representation(IR)** is a ***syntax tree*** in which **each interior node represents an operation** and the **children of the node represent the arguments of the operation.**
- **Syntax trees are high level representations; they depict the natural hierarchical structure of the source program and are well suited to tasks like static type checking.**

Phases of a compiler

2. Syntax Analysis

Example: Translation of a statement

$\langle \text{id}, 1 \rangle \Rightarrow \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$



We want a syntax tree as an intermediate representation because:

- It corresponds more closely to the meaning of the various program constructs (to the semantics).
- It is more compact.

Context Free Grammar(CFG):

$\text{AssignExpr} \rightarrow \text{id} = E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id} \mid \text{num} \mid (E)$

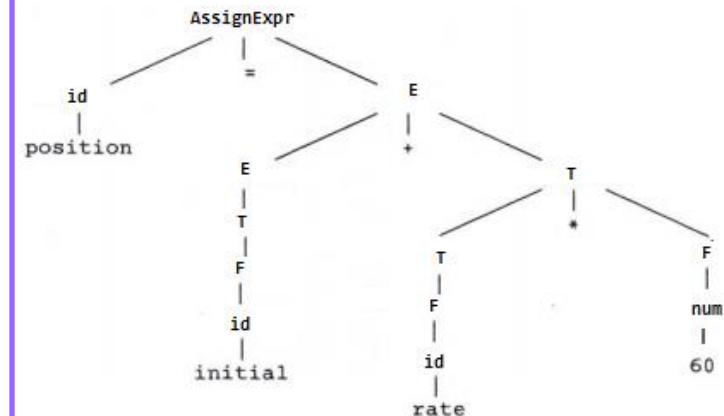


Figure Parse Tree for Position = initial + rate * 60

Parsing is the process of determining how a string of terminals can be generated by a grammar.

3. Semantic Analysis

- **Why have a Separate Semantic Analysis?**
 - Semantic consistency that cannot be handled at the parsing stage is handled here
 - Syntax Analysis/Parsing cannot catch some errors.

Phases of a compiler

3. Semantic Analysis

▣ Semantics

- ▣ Semantics of a language provide meaning to its constructs, like tokens and syntax structures.
- ▣ Semantics help interpret symbols, their types, and their relations with each other.

- ▣ **Semantic analysis judges whether the syntax structure constructed in the source program derives any meaning or not.** This is often implemented using Syntax-Directed Definitions or Syntax-Directed Translations.

CFG + semantic rules = Syntax Directed Definitions

3. Semantic Analysis

□ Example:

Semantic analysis judges whether the syntax structure constructed in the source program derives any meaning or not.

int a = “value”;

should not issue an error in lexical and syntax analysis phase, as it is lexically and structurally correct, but **Semantic analyzer should generate a semantic error as the type of the assignment differs.**

These rules are set by the grammar of the language and evaluated in semantic analysis.

3. Semantic Analysis

Static semantics (Compile-time checks)

- Rules that can be checked **at compile time** without running the program.
- Ensures the program statements are *well-formed* and meaningful before execution.
- **Examples:**
 - Type checking (e.g., you cannot assign a string to an integer variable).
 - Variable declaration before use.
 - Function call with correct number and type of arguments.
 - ...
- Implemented during **semantic analysis** phase of the compiler.

3. Semantic Analysis

Dynamic semantics

- ▣ Defines the **meaning of the program during execution** (runtime behavior).
- ▣ Specifies what happens when the program runs.
- ▣ **Examples:**
 - whether an overflow will occur during an arithmetic operation
 - whether array limits will be crossed during execution
 - whether recursion will cross stack limits
 - whether heap memory will be insufficient
 - ...

Phases of a compiler

3. Semantic Analysis

Static semantics check: Example

```
int dot_prod(int x[], int y[]){  
    int d, i; d = 0;  
    for (i=0; i<10; i++)  
        d += x[i]*y[i];  
    return d;  
}  
main(){  
    int p, a[10], b[10];  
    p = dot_prod(a,b);  
}
```

Samples of static semantic checks in *main* function.

- Types of p and return type of dot_prod match
- Number and type of the parameters of dot_prod are the same in both its declaration and use
- p is declared before use, same for a and b

Phases of a compiler

3. Semantic Analysis

Static semantics check: Example

```
int dot_prod(int x[], int y[]){  
    int d, i; d = 0;  
    for (i=0; i<10; i++)  
        d += x[i]*y[i];  
    return d;  
}  
main(){  
  
    int p, a[10], b[10];  
  
    p = dot_prod(a,b);  
  
}
```

Samples of static semantic checks in *dot_prod*

- d and i are declared before use
- Type of d matches the return type of dot_prod
- Type of d matches the result type of operation “*”
- Elements of arrays x and y are compatible with “*”

Phases of a compiler

3. Semantic Analysis

Dynamic semantics check

```
int dot_prod(int x[], int y[]){  
    int d, i; d = 0;  
    for (i=0; i<10; i++)  
        d += x[i]*y[i];  
    return d;  
}  
main(){  
  
    int p, a[10], b[10];  
  
    p = dot_prod(a,b);  
  
}
```

Samples of Dynamic semantic checks in *dot_prod*

- Value of **i** **does not exceed the declared range** of arrays x and y (both lower and upper)
- There are **no overflows** during the operations of “*” and “+” in **d += x[i]*y[i]**

Phases of a compiler

3. Semantic Analysis

➤ Static Semantics: Errors given by gcc Compiler

```
int dot_product(int a[], int b[]) {...}
```

```
1 main(){ int a[10]={1,2,3,4,5,6,7,8,9,10};  
2 int b[10]={1,2,3,4,5,6,7,8,9,10};  
3 printf("%d", dot_product(b));  
4 printf("%d", dot_product(a,b,a));  
5 int p[10]; p=dotproduct(a,b); printf("%d",p);}
```

3. Semantic Analysis

➤ Static Semantics: Errors given by gcc Compiler

```
int dot_product(int a[], int b[]) {...}
```

```
1  main(){ int a[10]={1,2,3,4,5,6,7,8,9,10};  
2  int b[10]={1,2,3,4,5,6,7,8,9,10};  
3  printf("%d", dot_product(b));  
4  printf("%d", dot_product(a,b,a));  
5  int p[10]; p=dotproduct(a,b); printf("%d",p);}
```

In function 'main':

```
error in 3: too few arguments to fn 'dot_product'  
error in 4: too many arguments to fn 'dot_product'  
error in 5: incompatible types in assignment  
warning in 5: format '%d' expects type 'int', but  
              argument 2 has type 'int *'
```

Phases of a compiler

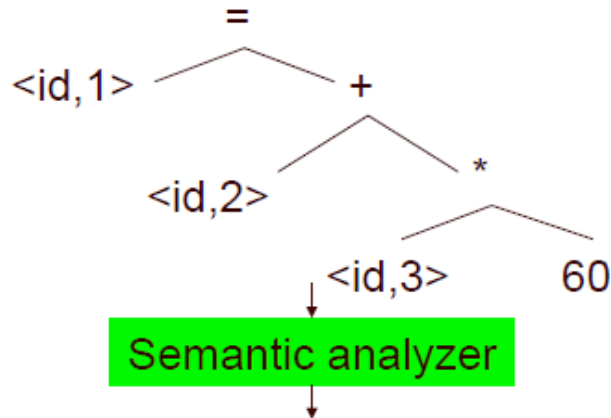
3. Semantic Analysis

- The semantic analyzer uses the syntax tree and the information in the **symbol table** to check the source program statements for semantic **consistency** with the language definition.
- The semantic analyzer **gathers type information and saves it in either the syntax tree or the symbol table**, for subsequent use during intermediate-code generation.
- An important part of semantic analysis is **type checking**, where the compiler **checks that each operator has matching operands**.
- The language specification may permit **coercions**. If mixing of types is allowed for binary arithmetic operator, the compiler may convert or **coerce the lower type to higher type** (int to float, float to double, ...)

Phases of a compiler

3. Semantic Analysis

□ Example: Translation of a statement



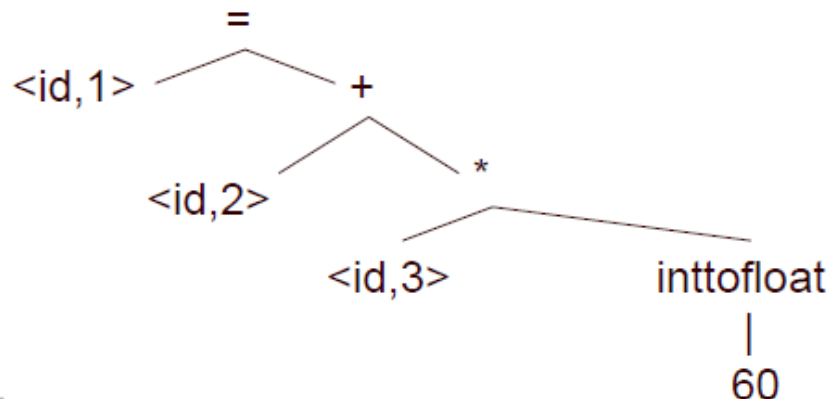
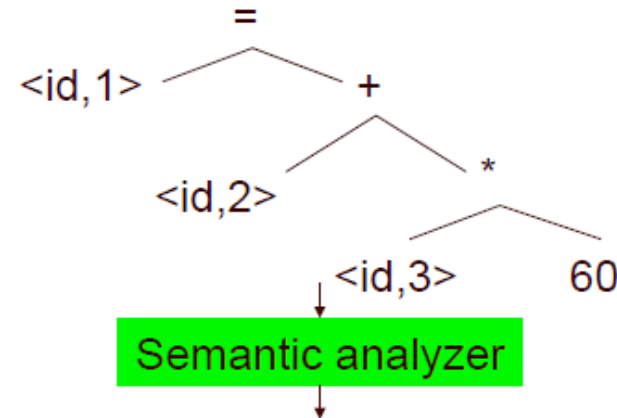
SYMBOL TABLE

1	position	FLOAT	
2	initial	FLOAT	
3	rate	FLOAT	
	...		

Phases of a compiler

3. Semantic Analysis

□ Example: Translation of a statement



SYMBOL TABLE

1	position	FLOAT	
2	initial	FLOAT	
3	rate	FLOAT	
	...		

3. Semantic Analysis

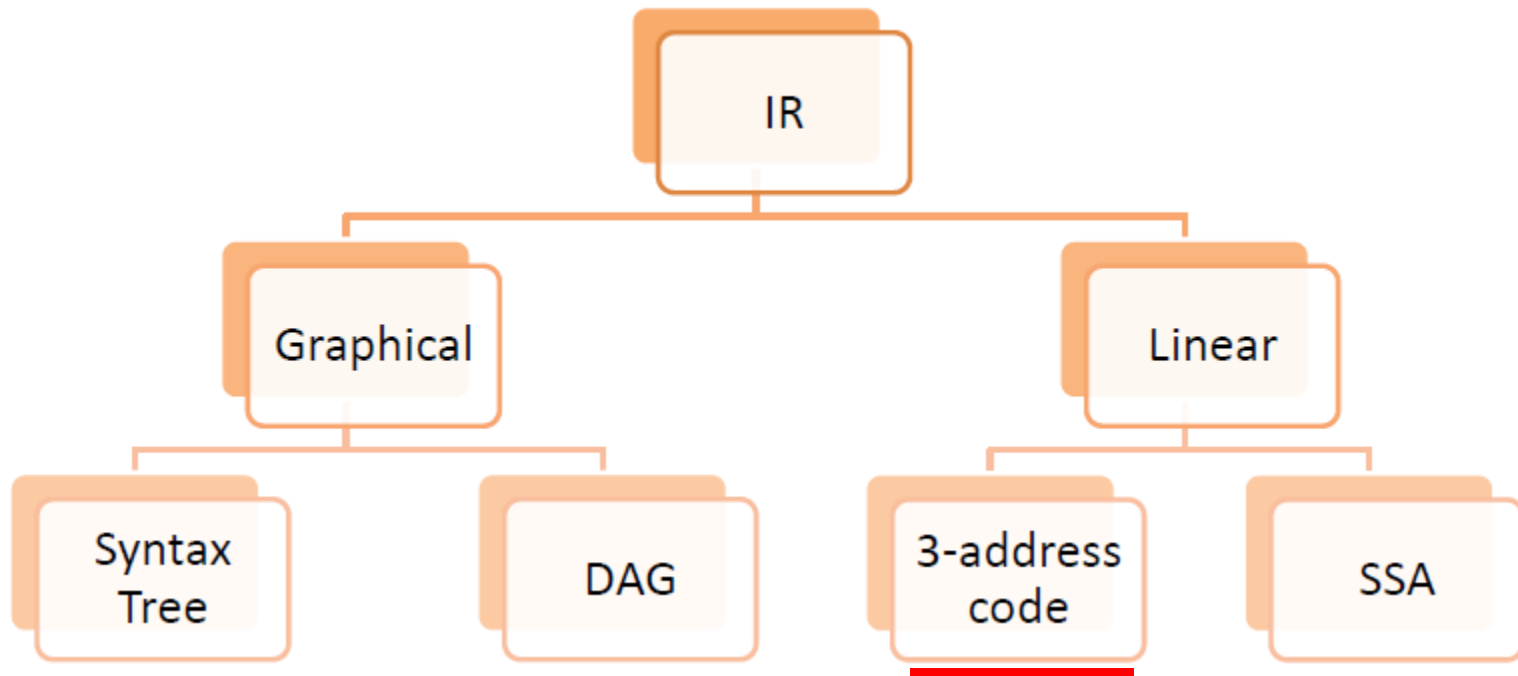
- Some of the **semantics errors** that the semantic analyzer is expected to recognize:
 - Type mismatch in computation
 - Undeclared variable
 - Reserved identifier misuse.
 - Multiple declaration of variable in a scope.
 - Accessing an out of scope variable.
 - Actual and formal parameter mismatch.
 -

4. Intermediate Code Generation

- ▣ After syntax and semantic analysis of the source program, **Compilers generate an explicit low-level or machine-like Intermediate Representation(IR), for an abstract machine.**
- ▣ The IR should have two important properties:
 1. it should be easy to produce and
 2. it should be easy to translate into the target machine.
- ▣ **IR is an abstract machine language**, that can express the target-machine operations, without committing too much machine-specific detail.

Phases of a compiler

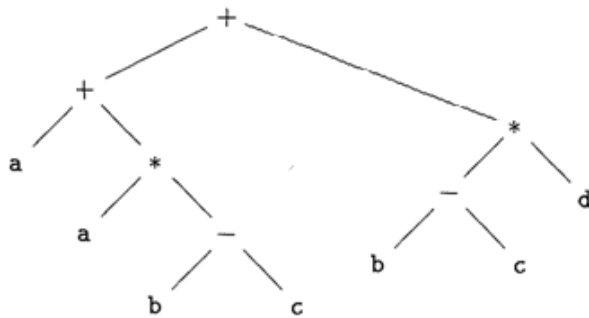
▣ Various Intermediate Representations (IR)



Phases of a compiler

Directed Acyclic Graph (DAG)

- A directed acyclic graph (DAG) for an expression identifies the common subexpressions (subexpressions that occur more than once) of the expression.
- The difference between DAG and Syntax tree
 - A node N in a DAG has more than one parent if N represents a common subexpression;
 - In a syntax tree, the tree for the common subexpression would be replicated as many times as the subexpression appears in the original expression.



syntax tree for the expression $a + a * (b - c) + (b - c) * d$

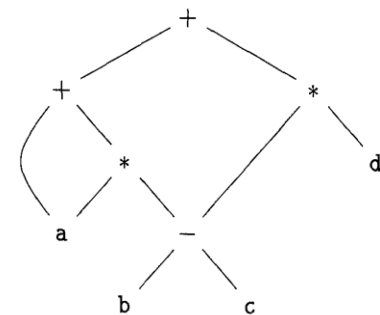


Figure 6.3: Dag for the expression $a + a * (b - c) + (b - c) * d$

Phases of a compiler

■ Three-Address Code(TAC)

■ In Three-Address Code,

- **each instruction has at most three operands(addresses).**
- **there is at most one operator on the right side of an instruction.**

■ Thus, a source-language expression like $x + y * z$ might be translated into the sequence of three-address instructions

$$\begin{aligned}t_1 &= y * z \\t_2 &= x + t_1\end{aligned}$$

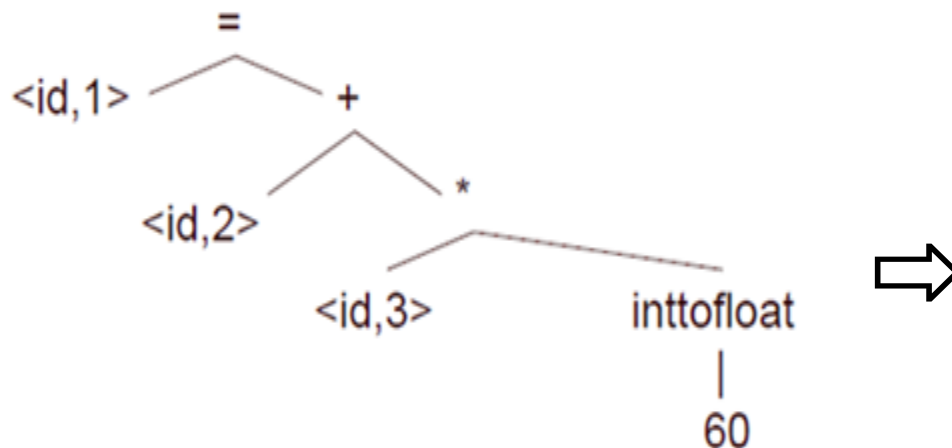
where t_1 and t_2 are compiler-generated temporary names.

The use of names for the intermediate values computed by a program allows three-address code to be rearranged easily.

Phases of a compiler

4. Intermediate Code Generation Cont...

- TAC consists of a sequence of assembly-like instructions with three operands per instruction. Each operand can act like a register.
- The output of the intermediate code generator in Fig.1.3 consists of the three-address code sequence.



Three Address Code (TAC):

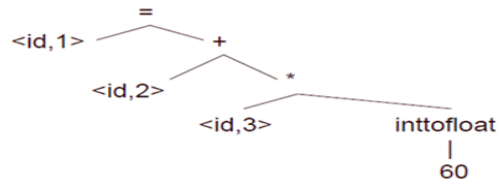
```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Figure:1.3

Phases of a compiler

4. Intermediate Code Generation Cont...

position = initial + rate * 60 (1.1)



t1 = inttofloat(60)

t2 = id3 * t1

t3 = id2 + t2

id1 = t3

1.3

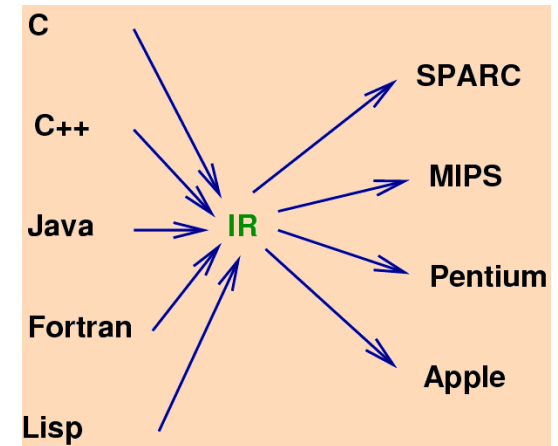
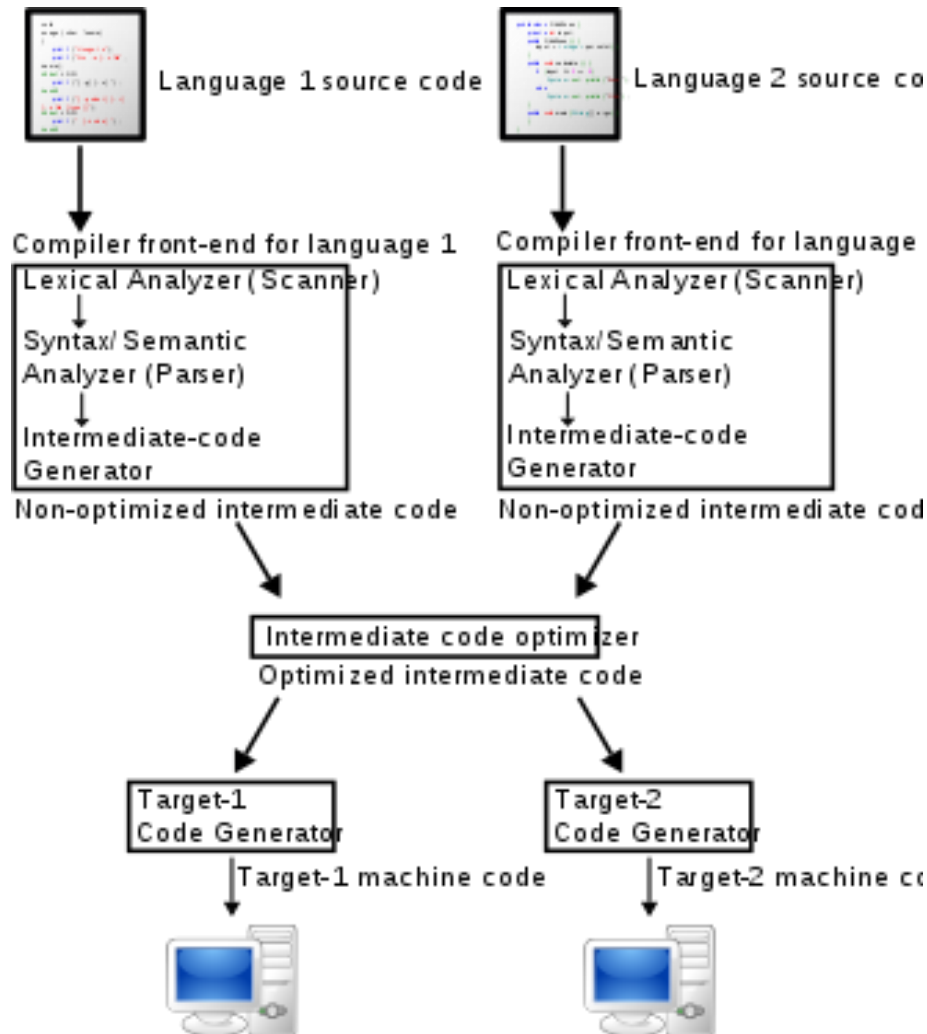
There are several points worth noting about three-address instructions.

- ❑ **First**, each three-address assignment instruction has at most one operator on the right side. Thus, **these instructions fix the order in which operations are to be done**. the multiplication precedes the addition in the source program (1.1).
- ❑ **Second**, the compiler must generate a **temporary name** to hold the value computed by a three-address instruction.
- ❑ **Third**, some "three-address instructions" like the first and last in the sequence (1.3), above, have fewer than three operands.

Three-address code is a linearized representation of a syntax tree.

Phases of a compiler

□ Why Intermediate code?



Goal: retargeting compiler components for different source languages/target machines.

5. Code Optimization (program transformation technique)

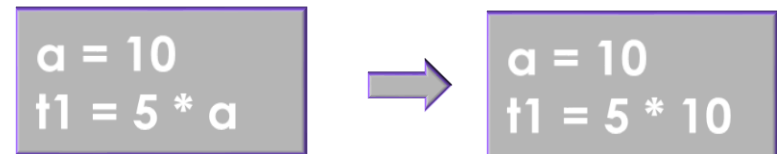
- Code optimization is an optional phase.
- The goal is to make the code run faster and use fewer resources while maintaining its functionality and correctness.
- Code optimization removes the unnecessary lines of code/computation and arranges the sequence of statements in order to speed up the program execution.

Phases of a compiler

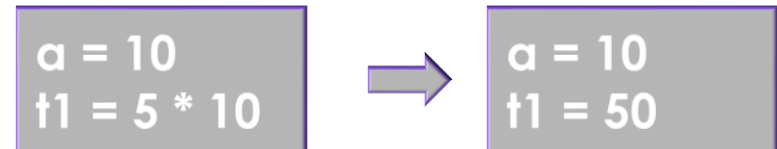
5. Code Optimization

Here are some techniques used in code optimization:

- **Constant propagation:** Substitutes a variable with its assigned constant.



- **Constant folding:** Computes variables whose values can be determined at compile time



- **Common sub expression elimination:** Eliminates common sub expressions
- **Dead code elimination:** Eliminates variables that are never used after their last definition or Eliminates statements that compute values that never get used.
- **Unreachable code elimination:** Eliminates unreachable code

Phases of a compiler

5. Code Optimization Cont...

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

(1.3) →

- The Code optimizer can deduce that the conversion of 60 from integer to floating point can be done once and for all at compile time, so the *inttofloat* operation can be eliminated by replacing the integer 60 by the floating-point number 60.0.
- Moreover, **t3** is used only once to transmit its value to **id1** so the optimizer can transform (1.3) into the shorter sequence

Phases of a compiler

5. Code Optimization Cont...

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

(1.3)

⇒

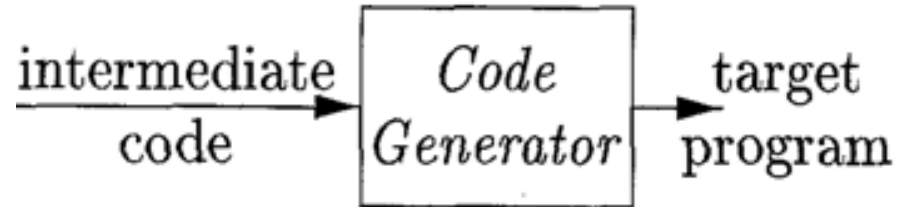
```
t1 = id3 * 60.0
id1 = id2 + t1
```

(1.4)

- The Code optimizer can deduce that the conversion of 60 from integer to floating point can be done once and for all at compile time, so the *inttofloat* operation can be eliminated by replacing the integer 60 by the floating-point number 60.0.
- Moreover, **t3** is used only once to transmit its value to **id1** so the optimizer can transform (1.3) into the shorter sequence

Phases of a compiler

6. Code Generation



- The code generator takes as input an intermediate representation of the source program and maps it into the target language.
- If the target language is machine code,
registers Or memory locations are selected for each of the variables used by the program.

Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.

Phases of a compiler

6. Code Generation

- ❑ A crucial aspect of code generation is **Instruction Selection** and the **judicious assignment of registers to hold variables**.
- ❑ For example, using registers R1 and R2, the intermediate code in (1.4) might get translated into the machine code

IR Code:

$t1 = id3 * 60.0$
(1.4)

$idl = id2 + t1$



Target Code:

```
LDF  R2, id3
MULF R2, R2, #60.0
LDF  R1, id2
ADDF R1, R1, R2
STF  idl, R1
```

- ❑ The **first operand** of each instruction specifies a destination.
- ❑ The **F** in each instruction opcode tells us that it deals with **floating-point numbers**.
- ❑ The **#** signifies that **60.0** is to be treated as an immediate constant.

Phases of a compiler

Symbol-Table Management

- ▣ The **symbol table is a data structure** containing a record for each identifier name, with fields for the attributes of the name.
- ▣ **Why symbol table?**
 - Used during all phases of compilation process.
 - It acts as a central repository for all information about identifiers and other language constructs.
 - Incrementally constructed and expanded during the analysis phases.
 - Used in the semantic and code generation phases.
 - Efficient storage and access is very important in symbol table.

Symbol-Table Management

Used During All Phases

- ▣ **Lexical Analysis:** When identifiers are first encountered, entries are created.
- ▣ **Syntax Analysis:** Adds more detailed attribute information to symbol table entries, like data type, scope (local/global), dimension (for arrays), and location.
- ▣ **Semantic Analysis:** Heavily uses the table for type-checking, scope resolution and verifying declarations.
- ▣ **Intermediate Code Generation:** The symbol table is consulted to determine memory requirements and runtime allocation. It also stores information for compiler-generated temporary variables created during this phase.
- ▣ **Code Generation:** Final addresses and storage details are retrieved from the symbol table.

Phases of a compiler

Symbol-Table Management cont...

- **An essential function of a compiler is to record the identifiers used in the source program and collect information about various attributes of each name.**

- **In the case of variable names,** the collected attributes may provide information about
 - the **storage (its size),**
 - its **type,**
 - its **scope.**
 - ...

Phases of a compiler

Symbol-Table Management cont...

- **In the case of procedure names**, the collected attributes may provide information about
 - the **number and types of its arguments**,
 - the **method of passing each argument** and
 - the **type returned**.
- **In the case of array names**, the collected attributes may provide information about
 - **number of Dimensions** and
 - **Array bounds**.
- **In the case of File names**, the collected attributes may provide information about **record size and record type**

Symbol-Table Management cont...

More about symbol tables

- ▣ Each piece of info associated with a name is called an attribute.
- ▣ Attributes are language dependent.
 - Actual Characters of the name
 - Type
 - Storage allocation info (number of bytes).
 - Line number where the symbol declared
 - Lines where referenced.
 - Scope

Phases of a compiler

Symbol-Table Management cont...

Other attributes:

- A scope of a variable can be represented by a number (level-1 scope, level-2 scope,)
- A different symbol table is constructed for different scope.

Symbol-Table Management cont...

- There are five main operations to be carried out on the symbol table:
 1. determining whether a name has already been stored
 2. inserting an entry for a name
 3. deleting a name when it goes out of scope
 4. to associate an attribute with a given entry
 5. to get an attribute associated with a given entry
- The above five operations requires the following functions:
 1. **lookup(s)**: returns the index of the entry for name s, or 0 if there is no entry
 2. **insert(s, t)**: add a new entry for name s (of token t), and return its index
 3. **delete(s)**: deletes s from the table (or, typically, hides it)
 4. **set_attribute(s, a)**: add an attribute a with a given entry s.
 5. **get_attribute(s)**: get an attribute associated with a given entry s.

Symbol-Table Management cont...

- The symbol table implementation mechanisms:
 - 1) Linear list (sorted or unsorted) and
 - 2) Hash table
 - 3) Binary search trees
- Each scheme is evaluated on the basis of time required to add n entries and make e inquiries.
- A **linear list** is the simplest to implement, but its performance is poor when n and e are large.
- **Hashing schemes** provide better performance for greater programming effort and space overhead, making them suitable for large programs.

Phases of a compiler

Symbol-Table Management cont...

□ Example: Linear list (2D-Array Implementation)

```
1. const int g=12;
2. float fun2(int i, float j) {
3.     int k, e;
4.     float z;
5.     e = 0;
6.     k = i * j + k;
7.     z = fun1(k, e);
8.     return z;
9. }
```

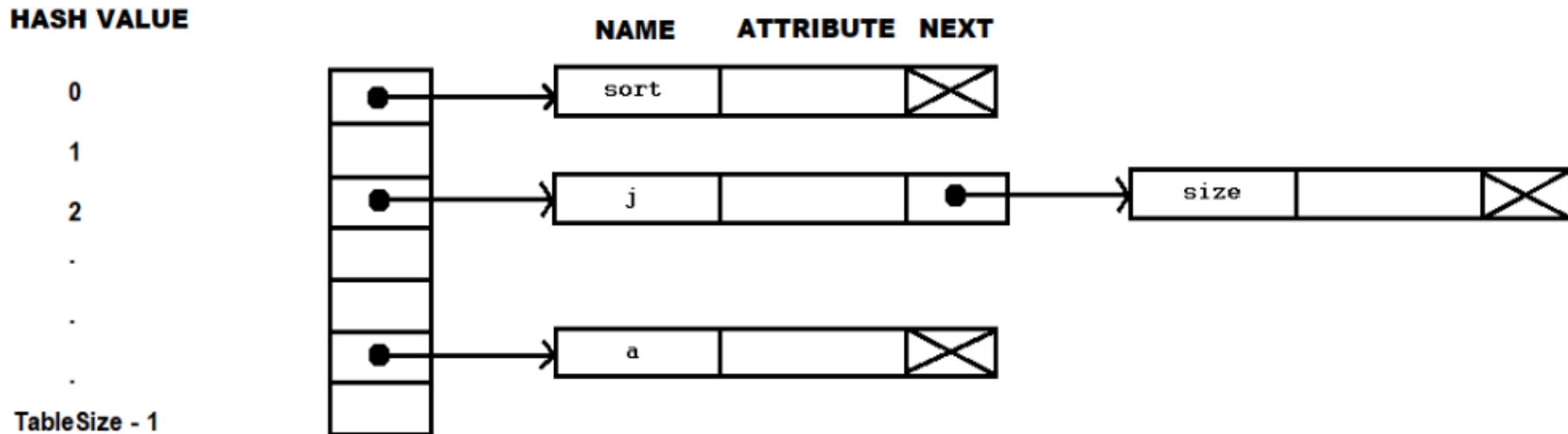
Name	type	value	size	scope	declared	referred	other
g	int	12	4	0	1		const
fun2					2		funname
i	int		4	1	2	6	parameter
j	float		4	1	2	6	parameter
k	int		4	1	3	6,7	variable
e	int	0	4	1	3	5,7	variable
z	float		4	1	4	7,8	variable
fun1						7	funname

Phases of a compiler

Symbol-Table Management cont...

□ Example: Hash table

The hash table is an array of pointers to linked lists called buckets



Phases of a compiler

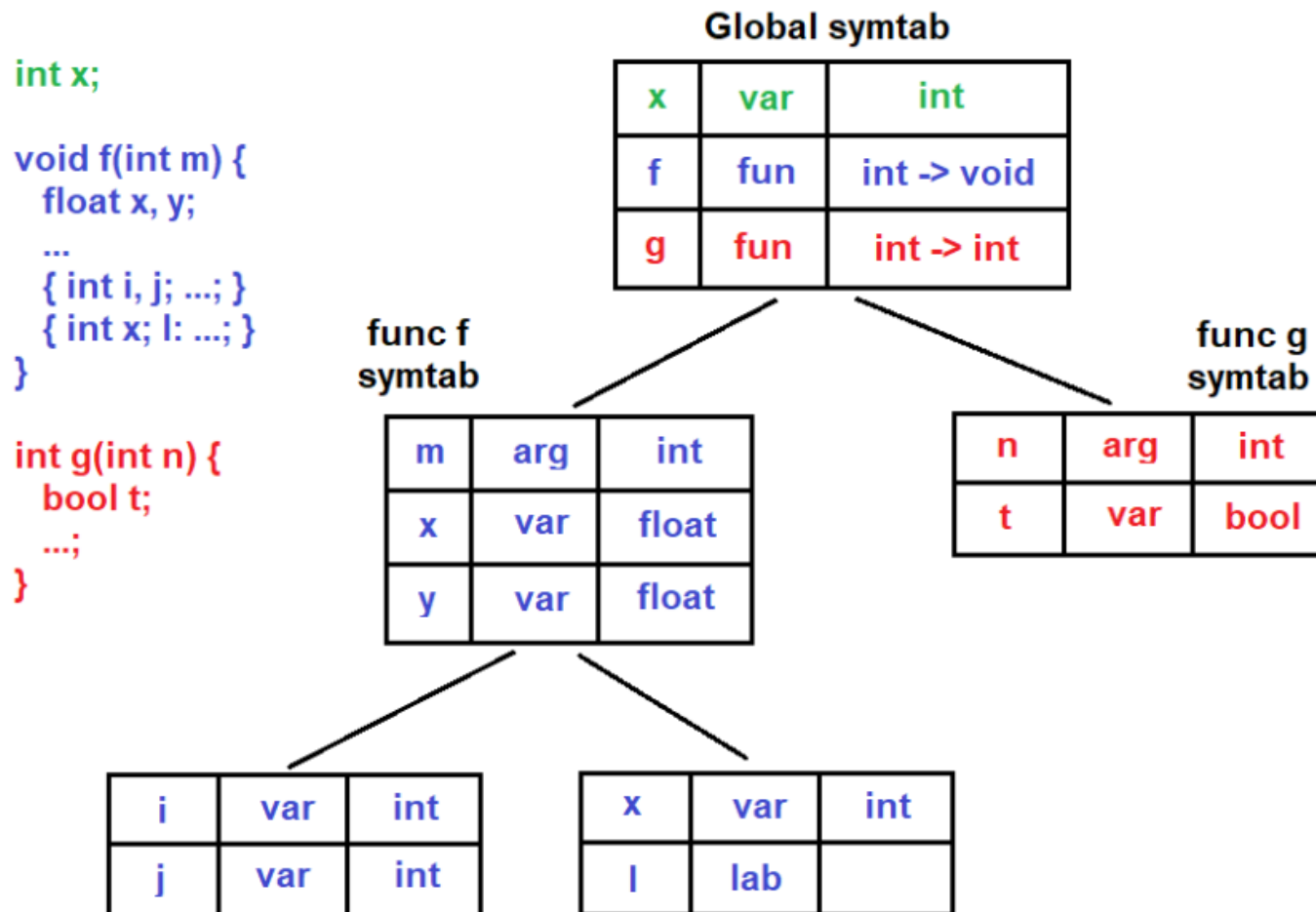
- Symbol tables typically need to support multiple declarations of the same identifier within a program.

Note: The scope of a declaration is the portion of a program to which the declaration applies.

- We shall implement scopes by setting up *a separate symbol table for each scope.*

Phases of a compiler

The following is an example of how scope information is represented in a symbol table:



Phases of a compiler

Exercise 1:

- Show the working of all the phases of Compiler on the given program statement:

```
while (y<z) {  
    int x = a + b;  
    y += x;  
}
```

- [Solution](#)

Phases of a compiler

Exercise 2:

- Show the working of all the phases of Compiler on the given program statement:

```
/* code in C language */  
  
do{  
    fact = fact * counter;  
    counter--;  
}while(counter > 0);
```

References

- ▣ **Compilers–Principles, Techniques and Tools**, Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffery D. Ullman, 2nd Edition



THANK YOU

Prakash C O

Associate Professor,

Department of Computer Science & Engineering.

coprakasha@pes.edu